

Aula 02 — Do procedural aos objetos

Como tornar explícitas no código as unidades que já reconhecemos no problema?

No Laboratório 01...

```
String descricao1 = "Teclado";  
double preco1 = 150.0;  
int quantidade1 = 2;
```

```
String descricao2 = "Mouse";  
double preco2 = 80.0;  
int quantidade2 = 3;
```

PROBLEMA INICIAL

Quando os itens aumentaram...

- O que começou a ficar difícil?
- Quais variáveis representam partes de uma mesma coisa?
- Como manter dados e operações relacionados mais próximos?

Os dados estão relacionados

Item 1

descricao1

preco1

quantidade1

Item 2

descricao2

preco2

quantidade2

Reconhecemos dois itens no problema, mesmo que seus dados estejam separados no código.

PROBLEMA INICIAL

**Se descrição, preço e quantidade pertencem ao mesmo item,
onde está o Item no nosso programa?**

PROBLEMA INICIAL

Uma possível evolução procedural

```
String[] descricoes = {"Teclado", "Mouse"};  
double[] precos = {150.0, 80.0};  
int[] quantidades = {2, 3};
```

PROBLEMA INICIAL

O que melhorou?

E o que ainda depende de uma convenção?

```
descricoes[i]  
precos[i]  
quantidades[i]
```

PROBLEMA INICIAL

A posição mantém a relação

posição 0

"Teclado"
150.0
2

posição 1

"Mouse"
80.0
3

Os arrays reduzem a repetição, mas a relação entre os dados ainda depende do mesmo índice nas três estruturas.

PRIMEIROS OBJETOS

O Item existe no problema

Mas ainda está implícito no código.

PRIMEIROS OBJETOS

Tornando a unidade explícita

ItemPedido

descrição
preço unitário
quantidade

CONCEITO-CHAVE

PRIMEIROS OBJETOS

Objeto

Uma unidade da solução que reúne informações e operações relacionadas.

O estado de um item

ItemPedido

```
descricao = "Teclado"  
precoUnitario = 150.0  
quantidade = 2
```

Estado: informações que caracterizam o objeto em determinado momento.

PRIMEIROS OBJETOS

Um objeto apenas guarda dados?

O que esse item precisa saber fazer com o próprio estado?

Comportamento

```
calcularSubtotal()
```

preço unitário × quantidade → subtotal

Comportamento: uma operação relacionada ao objeto que pode usar seu estado.

Estado + comportamento

Estado

descrição
preço unitário
quantidade

Comportamento

`calcularSubtotal()`

`ItemPedido` representa uma unidade relevante para esta solução.

PRIMEIROS OBJETOS

Todo substantivo vira classe?

O nome é apenas uma pista

ItemPedido importa porque possui estado, comportamento e um papel coerente na solução.

O nome ajuda a perceber um conceito, mas não decide sozinho quais classes devem existir.

CONCEITO-CHAVE
DA IDEIA AO JAVA

Classe

Define a estrutura e os comportamentos comuns aos objetos daquele tipo.

Primeira representação em Java

```
public class ItemPedido {  
  
    String descricao;  
    double precoUnitario;  
    int quantidade;  
}
```

class ItemPedido declara a classe; cada campo informa um tipo e um nome.

DICA

DA IDEIA AO JAVA

Mantenha os arquivos no mesmo projeto

```
lab-02/  
├── Main.java  
└── ItemPedido.java
```

Abra a pasta do projeto na IDE, não apenas os arquivos separadamente.

A estrutura agora está explícita

ItemPedido

```
String descricao  
double precoUnitario  
int quantidade
```

A classe reúne a estrutura comum dos itens.

Criando o primeiro objeto

```
ItemPedido item1 = new ItemPedido();  
  
item1.descricao = "Teclado";  
item1.precoUnitario = 150.0;  
item1.quantidade = 2;
```

`new ItemPedido()` cria um novo objeto do tipo `ItemPedido`. Os parênteses pertencem ao mecanismo de construtores; esse assunto fica adiado por enquanto.

Estado do objeto usado por `item1`

objeto `ItemPedido`

```
descricao = "Teclado"  
precoUnitario = 150.0  
quantidade = 2
```

Criando outro objeto

```
ItemPedido item2 = new ItemPedido();  
  
item2.descricao = "Mouse";  
item2.precoUnitario = 80.0;  
item2.quantidade = 3;
```


Uma classe, vários objetos

- Quantas classes aparecem?
- Quantos objetos foram criados?
- Os objetos precisam possuir o mesmo estado?

Estrutura comum, estados próprios

objeto usado por `item1`

"Teclado"
150.0
2

objeto usado por `item2`

"Mouse"
80.0
3

Uma classe pode originar vários objetos com estados diferentes.

Acrescentando comportamento

```
public class ItemPedido {  
    String descricao;  
    double precoUnitario;  
    int quantidade;  
    double calcularSubtotal() {  
        return precoUnitario * quantidade;  
    }  
}
```

`double` → valor retornado `calcularSubtotal` → nome `()` → parâmetros `{ ... }` → corpo `return` → valor devolvido

RESPONSABILIDADE

Primeira possibilidade

```
item.precoUnitario * item.quantidade
```

Quem está realizando esse cálculo?

RESPONSABILIDADE

O código externo faz o cálculo

acessa o preço



acessa a quantidade



calcula

RESPONSABILIDADE

Segunda possibilidade

```
calcularSubtotal(item)
```

Onde ficou a responsabilidade pelo cálculo?

Um método auxiliar recebe o item

```
double calcularSubtotal(ItemPedido item) {  
    return item.precoUnitario * item.quantidade;  
}
```

O cálculo foi nomeado, mas o método ainda precisa receber o item e acessar seu estado.

RESPONSABILIDADE

Terceira possibilidade

```
item.calcularSubtotal()
```

Quem já possui os dados necessários?

RESPONSABILIDADE

O próprio objeto já possui os dados

- `precoUnitario`
- `quantidade`

O objeto pode usar seu próprio estado para calcular o subtotal.

Métodos podem produzir resultados ou ações

```
double calcularSubtotal() {  
    return precoUnitario * quantidade;  
}
```

calcula e devolve um valor `double`

```
void aumentarQuantidade(int unidades) {  
    // ação sobre o estado  
}
```

recebe um valor, pode alterar o estado e não devolve um resultado

`void` indica que o método executa uma ação sem devolver um valor. A lógica da alteração ainda precisa ser construída.

CONCEITO-CHAVE

RESPONSABILIDADE

Responsabilidade

Responsabilidade indica qual objeto deve cuidar de determinado estado ou comportamento na solução.

Atividade em dupla — Produto

```
class Produto {  
    String nome;  
    double preco;  
  
    double calcularPrecoComDesconto(double percentual) {  
        return preco - preco * percentual;  
    }  
}
```

Expliquem o código

1. O que a classe representa?
2. Quais elementos formam o estado?
3. Qual comportamento aparece?
4. Objetos diferentes precisam ter o mesmo preço?
5. Por que o método acessa `preco` sem recebê-lo?

Discussão da atividade

Estado

nome

preco

Comportamento

```
calcularPrecoComDesconto(...)
```

O método acessa `preco` porque utiliza o estado do próprio objeto.

Transferência — biblioteca

Um empréstimo registra:

- o livro;
- a data;
- se já foi devolvido.

Um empréstimo pode ser devolvido.

Qual estado, comportamento e responsabilidade aparecem?

Uma possível modelagem

Estado de Empréstimo

livro
data
situação da devolução

Comportamento

registrar a devolução

O importante é justificar o conceito por seu estado, comportamento e papel na solução.

SÍNTESE
SÍNTESE

Da solução procedural aos objetos

dados relacionados

→

objeto explícito

→

estado + comportamento

→

responsabilidades

O que precisamos conseguir explicar

- Por que representar explicitamente um `ItemPedido` ?
- O que são objeto, estado, comportamento e classe neste exemplo?
- Como uma classe pode originar vários objetos?
- Por que `calcularSubtotal()` pode pertencer a `ItemPedido` ?

Código que você não consegue explicar não é código que você domina.

SÍNTESE

Hoje

Identificamos uma unidade que estava implícita na solução procedural e a representamos como objeto.

PRÓXIMO PASSO

No Laboratório 02, vamos transformar a solução construída anteriormente usando essa nova organização.
